SAE-2.03 Serveur Web

Gallois Serlooten Wagner Pierre

Le Madec Jean

Group : E

Diagramme Class :



Description des Méthodes

Main :

La méthode main(String[] args) constitue le point d'entrée du programme et est responsable de l'initialisation globale du serveur web. Tout d'abord, elle récupère l'identifiant du processus Java en cours d'exécution grâce à la classe ManagementFactory. Cet identifiant est ensuite écrit dans un fichier ../run/myweb.pid, ce qui permet de garder une trace du processus serveur et facilite son administration ou son arrêt.

Ensuite, la méthode charge les fichiers de configuration situés dans le dossier ../conf.d. Ces fichiers, au format XML, décrivent les différents sites web à héberger. Si aucun fichier de configuration n'est trouvé, le programme affiche un message d'erreur et s'arrête immédiatement. Chaque fichier .conf est ensuite analysé à l'aide d'un parseur XML (DocumentBuilder de javax.xml) afin d'extraire les informations de chaque balise <site>, notamment le dossier racine du site (DocumentRoot), la page par

défaut (DefaultIndex), les chemins des fichiers de log (Acceslog et Errorlog), ainsi que le port d'écoute du serveur.

Ces informations sont stockées dans une structure de type TreeMap, associant chaque port à la configuration correspondante du site. Cela permet de gérer plusieurs sites web simultanément sur des ports différents. Enfin, pour chaque port configuré, la méthode crée un thread indépendant qui lance la méthode chargerPage(...). Ainsi, chaque site web fonctionne de manière autonome et en parallèle.

chargerPage :

Paramètres :

pathRoot - dossier racine contenant les fichiers du site

port - port réseau sur lequel le serveur écoute (ex : 80, 8080)

pathDefault - nom du fichier à charger par défaut dans un dossier

accessLogPath - chemin du fichier de log des accès réussis

errorLogPath - chemin du fichier de log des erreurs (404, 403...)

Lève :

IOException si la création du serveur sur ce port échoue

La méthode chargerPage est la méthode centrale du serveur. Elle ouvre un ServerSocket sur le port donné et tourne en boucle infinie pour traiter les requêtes HTTP entrantes. Pour chaque requête reçue :

- Elle lit et analyse la ligne de requête HTTP pour extraire le chemin du fichier demandé.
- Elle lit les en-têtes du navigateur : If-None-Match (ETag), If-Modified-Since et Accept-Encoding (pour Gzip).
- Elle gère la page spéciale /status qui affiche la mémoire JVM disponible, l'espace disque libre sur / et le nombre de processus Linux (via ps -e).
- Elle vérifie la sécurité en bloquant tout chemin contenant .. (attaque Path Traversal) et renvoie une erreur 403 dans ce cas.
- Elle supprime le query string (?param=val) si présent dans l'URL.
- Si le fichier demandé n'existe pas, elle renvoie une erreur 404.
- Si le chemin pointe vers un dossier sans fichier par défaut, elle génère et envoie un listing HTML du contenu du répertoire avec des liens cliquables.
- Elle gère le cache HTTP en comparant l'ETag calculé avec celui du client ; si les versions correspondent, elle renvoie un code 304 (Not Modified) sans renvoyer le fichier.
- Pour les fichiers HTML, elle appelle traiterHtmlDynamique avant l'envoi.
- Elle compresse les images (PNG, JPG, GIF) et les PDF en Gzip si le client l'accepte.
- Elle envoie les en-têtes HTTP (Date, Server: MyWeb/1.0, Content-Type, Content-Length, ETag, Last-Modified).
- Elle enregistre chaque accès et chaque erreur dans les fichiers de log via logToFile.

traiterHtmlDynamique

Paramètre :

fichier - le fichier HTML contenant les balises dynamiques

Retourne :

le contenu HTML modifié sous forme de tableau d'octets (byte[])

Lève :

IOException en cas de problème de lecture du fichier

La méthode traiterHtmlDynamique(File fichier) analyse les fichiers HTML contenant du code exécutable côté serveur. Elle commence par lire l'intégralité du fichier HTML, puis utilise une expression régulière pour repérer les balises <code interpreteur="...">. La regex a été conçue pour accepter différents types de guillemets (", ', «») afin d'être compatible avec divers éditeurs de texte. Pour chaque balise trouvée, elle extrait l'interpréteur (par exemple /bin/bash ou /usr/bin/python3) ainsi que le code à exécuter, appelle executerScript et remplace la balise par le résultat. La méthode retourne le contenu HTML final sous forme de tableau d'octets encodé en UTF-8.

executerScript :

Paramètres :

interpreteur - chemin de l'interpréteur (ex : /bin/bash, /usr/bin/python3)

code - le code ou la commande à faire exécuter

Retourne :

le texte produit par le script (stdout), ou un message d'erreur entre crochets

La méthode executerScript(String interpreteur, String code) permet d'exécuter un script via un interpréteur système. Elle utilise ProcessBuilder pour lancer un processus externe avec le paramètre -c (qui permet de passer le code directement en argument). Elle récupère ensuite la sortie standard du processus ligne par ligne via un BufferedReader, attend la fin du processus avec waitFor(), et retourne le résultat sous forme de chaîne de caractères. En cas d'erreur, elle retourne un message entre crochets plutôt que de planter le serveur.

logToFile :

Paramètres :

cheminLog - chemin du fichier de log

message - le message à enregistrer (accès, erreur, etc.)

La méthode logToFile(String cheminLog, String message) est responsable de l'enregistrement des événements du serveur. Si le chemin est null ou vide, elle ne fait rien. Sinon, elle crée automatiquement les répertoires nécessaires avec mkdirs(), puis ouvre le fichier en mode ajout (append = true) et écrit la date et l'heure courantes suivies du message. Cette méthode est appelée pour tracer à la fois les accès réussis (200, 304) et les erreurs rencontrées (404, 403).

calculerMD5 :

Paramètre :

fichier - le fichier dont on veut calculer l'empreinte

Retourne :

l'empreinte MD5 en hexadécimal entre guillemets, ou null en cas d'erreur

La méthode calculerMD5(File fichier) calcule une empreinte numérique unique d'un fichier grâce à l'algorithme MD5, via la classe MessageDigest. Elle lit le fichier par blocs de 1024 octets, met à jour le hash à chaque bloc, puis produit la signature finale. Le résultat est converti en chaîne hexadécimale et entouré de guillemets (comme le requiert la norme HTTP pour un ETag). Cette empreinte permet au navigateur de savoir si le fichier a changé depuis sa dernière visite, et d'éviter un téléchargement inutile.

formaterDate :

Paramètre :

timestamp - temps en millisecondes depuis le 1er janvier 1970

Retourne :

la date formatée en chaîne de caractères pour les en-têtes HTTP

La méthode formaterDate(long timestamp) convertit un horodatage en une chaîne de caractères au format HTTP standard (RFC 1123), avec le fuseau horaire forcé en GMT via SimpleDateFormat. Elle est utilisée pour renseigner les en-têtes HTTP Date et Last-Modified dans les réponses envoyées au client. Par exemple : Tue, 10 Jun 2026 12:00:00 GMT.

doitEtreZippe :

Paramètre :

nomFichier - le nom du fichier à vérifier

Retourne :

true si le fichier doit être compressé, false sinon

La méthode doitEtreZippe(String nomFichier) détermine si un fichier doit être compressé en Gzip avant d'être envoyé au client. Conformément aux consignes de la SAE, elle cible les images (PNG, JPG, JPEG, GIF) et les documents PDF. Son résultat est utilisé dans chargerPage avec la vérification que le navigateur accepte Gzip (Accept-Encoding: gzip).

compresserGZip :

Paramètre :

donnees - les données brutes à compresser

Retourne :

le tableau d'octets compressé

Lève :

IOException si une erreur survient pendant la compression

La méthode compresserGZip(byte[] donnees) compresse un tableau d'octets en utilisant l'algorithme GZIP, via la classe GZIPOutputStream écrivant dans un ByteArrayOutputStream. Elle retourne les données compressées sous forme de tableau d'octets, permettant de réduire la taille des fichiers transmis au navigateur.

getContentType :

Paramètre :

nomFichier - le nom ou le chemin du fichier

Retourne :

le type MIME correspondant (ex : "text/html", "image/png")

La méthode getContentType(String nomFichier) retourne le type MIME correspondant à l'extension du fichier demandé. Elle compare l'extension en minuscules pour être insensible à la casse. Exemples de

correspondances : .html → text/html; charset=UTF-8, .css → text/css, .js → application/javascript, .png → image/png, .pdf → application/pdf. Si l'extension n'est pas reconnue, la méthode retourne application/octet-stream par défaut. Ce type est inclus dans l'en-tête Content-Type de la réponse HTTP afin que le navigateur sache comment interpréter le fichier reçu.

Arborescence Serveur :

Version texte : ServerWeb_GALLOIS_LE_MADEC

```
├── arborescenceServeur.png
├── arborescenceServeur.puml
├── binome.txt
├── myweb
│   ├── DEBIAN
│   │   └── control
│   ├── lib
│   │   └── systemd
│   │       └── system
│   │           └── serverWeb.service
│   └── opt
│       └── serveurWeb
│           ├── bin
│           │   ├── HttpServer$1.class
│           │   └── HttpServer.class
│           ├── conf.d
│           │   ├── serverWeb.conf
│           │   └── siteTest.conf
│           ├── run
│           └── scripts
│               ├── start.sh
│               └── stop.sh
├── myweb.deb
├── README.md
└── serveurWeb
    ├── bin
    │   ├── HttpServer$1.class
    │   ├── HttpServer.class
    │   └── HttpServer.java
    ├── conf.d
    │   ├── serverWeb.conf
    │   └── siteTest.conf
    ├── run
    │   └── serverWeb.service
    ├── scripts
    │   ├── start.sh
    │   └── stop.sh
    └── var
        ├── log
        │   ├── site-1
        │   │   ├── access.log
        │   │   └── error.log
        │   ├── site-2
        │   │   ├── access.log
        │   │   └── error.log
        │   └── site-3
        │       ├── access.log
        │       └── error.log
        ├── site-1
        │   ├── images
        │   │   ├── l1.gif
        │   │   └── n2.jpg
        │   ├── index.html
        │   ├── miniWeb.html
        │   ├── technique.html
        │   └── test.png
        ├── site-2
        │   ├── demo.html
        │   ├── dossier_vide
        │   ├── miniWeb.html
        │   └── style.css
        └── site-3
            ├── images
            │   ├── l1.gif
            │   └── n2.jpg
            ├── index.html
            ├── miniWeb.html
            ├── technique.html
            └── test.png
```

